

Zbývající problémy v modulu tikhonov.c

Dokument: Analýza kódu po opravách V4.7

Datum: 2025-11-28

Stav: Kritické chyby opraveny, zbývají střední a nízké priority

Přehled

Kategorie	Opraveno	Zbývá
Kritické logické chyby	3/3	0
Závažné strukturální chyby	2/2	0
Numerické problémy	1/3	2
Kvalita kódu	0/2	2
Možná vylepšení	1/4	3

1. Numerické problémy

1.1 Trace(H) aproximace platí pouze pro uniformní mřížky

Závažnost: Střední

Umístění: compute_gcv_score_robust(), řádky ~360-375

Popis problému:

Výpočet trace(H) pro GCV kritérium používá analytický vzorec odvozený pro **uniformní mřížku**:

```
for (int k = 1; k <= n-2; k++) {  
    double theta = M_PI * k / n;  
    double eigenval = 4.0 * pow(sin(theta/2.0), 2) / (h_avg * h_avg);  
    trace_H += 1.0 / (1.0 + lambda * eigenval);  
}
```

Tento vzorec předpokládá, že vlastní čísla matice K jsou $4 \sin^2(\pi k/2n) / h^2$, což platí **pouze** pro uniformní mřížku s konstantním krokem h.

Důsledky:

- Pro neuniformní mřížky je trace(H) nepřesná
- GCV skóre je zkreslené → optimální λ může být suboptimální
- Kód sice vypisuje varování pro $\text{ratio} > 2.0$, ale pokračuje s nepřesným výpočtem

Možná řešení:

1. **Stochastická aproximace trace:** Použít Hutchinsonův estimátor

`// trace(A)  $\approx$  zT A z, kde z je náhodný vektor  $\pm 1$`

2. **Diagonální aproximace:** Spočítat přímo diagonální prvky $(A + \lambda K)^{-1}$

3. Robustní alternativa: Pro neuniformní mřížky použít L-curve místo GCV

1.2 Potenciální dělení nulou

Závažnost: Nízká (teoretická)

Umístění: `build_band_matrix()`, `compute_functional()`, `compute_derivatives()`

Popis problému:

Kód dělí hodnotami `h_left`, `h_right`, `h_sum` bez explicitní kontroly:

```
double w = 2.0 * lambda / h_sum;
AB[kd + j*ldab] += w * (1.0/h_left + 1.0/h_right);
```

Současná ochrana:

Hlavní funkce `tikhonov_smooth()` validuje monotonnost:

```
for (int i = 1; i < n; i++) {
    if (x[i] <= x[i-1]) {
        fprintf(stderr, "Error: x array must be strictly increasing\n");
        return NULL;
    }
}
```

Zbývající riziko:

- Statické funkce neprovádějí vlastní validaci
- Extrémně malé hodnoty h (např. $1e-300$) mohou způsobit overflow při $1/h^2$
- Při budoucím refaktoringu může být validace omylem odstraněna

Doporučení:

Přidat defenzivní kontrolu s minimálním krokem:

```
#define H_MIN_SAFE 1e-15
if (h_left < H_MIN_SAFE || h_right < H_MIN_SAFE) {
    fprintf(stderr, "Error: Grid spacing too small\n");
    return NULL; // nebo graceful degradation
}
```

2. Kvalita kódu

2.1 Magic numbers bez dokumentace

Závažnost: Nízká

Umístění: Celý modul

Seznam nedokumentovaných konstant:

Hodnota	Místo	Předpokládaný účel
0.15	CV_THRESHOLD	Hranice pro výběr diskretizační metody
0.7	compute_gcv_score	Práh pro penalizaci vysokého trace(H)
10.0	compute_gcv_score	Síla exponentní penalizace
5000	compute_gcv_score	Přibližná rychlost na rychlou aproximaci
20000	find_optimal_lambda_gcv	Práh pro "velký dataset"
1e-6, 1e0	find_optimal_lambda_gcv	Rozsah hledání λ
0.3, 1.7	refinement loop	Faktory pro zjemnění hledání

Doporučení:

Definovat jako pojmenované konstanty s komentářem:

```
/* Threshold for switching discretization methods.
 * Based on empirical testing: CV < 0.15 indicates near-uniform grid
 * where average coefficient method is sufficiently accurate. */
#define CV_THRESHOLD 0.15

/* GCV trace penalty threshold.
 * When trace(H)/n > 0.7, the smoother is overfitting.
 * Exponential penalty discourages such solutions. */
#define GCV_TRACE_PENALTY_THRESHOLD 0.7
#define GCV_TRACE_PENALTY_STRENGTH 10.0
```

2.2 Oddělení I/O od výpočetní logiky

Závažnost: Nízká (architektonická)

Umístění: find_optimal_lambda_gcv(), compute_gcv_score_robust()

Popis problému:

Funkce přímo tisknou na stdout:

```
printf("# GCV optimization for n=%d points\n", n);
printf("# λ=%9.3e: J=%9.3e, RSS=%9.3e, tr(H)=%6.1f...\n", ...);
```

Důsledky:

- Nelze použít v GUI aplikacích nebo serverovém kódu
- Obtížné unit testování
- Nelze přeměrovat výstup bez hackování

Doporučení:

Zavést callback nebo strukturu pro diagnostiku:

```
typedef struct {
    int verbose_level;
    FILE *output_stream;
    // nebo callback:
```

```

    void (*log_callback)(const char *msg, void *user_data);
    void *user_data;
} TikhonovOptions;

double find_optimal_lambda_gcv(double *x, double *y, int n,
                               GridAnalysis *grid_info,
                               TikhonovOptions *options);

```

3. Možná vylepšení

3.1 Vyšší přesnost derivací na hranicích

Umístění: compute_derivatives()

Současný stav:

```

/* Forward difference at start - O(h) */
y_deriv[0] = (y_smooth[1] - y_smooth[0]) / (x[1] - x[0]);

/* Backward difference at end - O(h) */
y_deriv[n-1] = (y_smooth[n-1] - y_smooth[n-2]) / (x[n-1] - x[n-2]);

```

Problém:

Jednostranné difference prvního řádu mají chybu $O(h)$, zatímco centrální difference ve vnitřních bodech mají $O(h^2)$.

Vylepšení:

Použít jednostranné formule druhého řádu:

```

/* Forward difference O(h^2) - requires 3 points */
if (n >= 3) {
    double h0 = x[1] - x[0];
    double h1 = x[2] - x[1];
    y_deriv[0] = (-(2*h0 + h1)*y[0] + (h0 + h1)*(h0 + h1)*y[1]/(h0*h1)
                  - h0*h0*y[2]/(h1*(h0 + h1))) / h0;
}

/* Nebo jednodušší pro uniformní mřížku: */
y_deriv[0] = (-3*y[0] + 4*y[1] - y[2]) / (2*h);

```

3.2 Validace vstupů v pomocných funkcích

Popis:

Statické funkce (build_band_matrix, compute_derivatives, atd.) spoléhají na validaci v hlavní funkci. Pro robustnost a snazší debugging by měly mít vlastní asserty:

```

#include <assert.h>

static void build_band_matrix(double *x, int n, double lambda,
                             double *AB, int ldab, int kd,
                             GridAnalysis *grid_info)
{
    assert(x != NULL);
    assert(AB != NULL);
    assert(n >= 1);
    assert(ldab >= kd + 1);
    assert(lambda >= 0.0);

    // ... zbytek funkce
}

```

3.3 Podpora pro volitelné váhy datových bodů

Popis:

Současná implementace řeší:

$$\min_u \|y - u\|^2 + \lambda \|D^2 u\|^2$$

Obecnější formulace s váhami:

$$\min_u \sum_i w_i (y_i - u_i)^2 + \lambda \|D^2 u\|^2$$

Využití:

- Různá důvěryhodnost měření
- Robustní regrese (iterativně převážené nejmenší čtverce)
- Heteroskedastická data

Implementace:

```

TikhonovResult* tikhonov_smooth_weighted(
    double *x, double *y, double *weights, // weights can be NULL
    int n, double lambda, GridAnalysis *grid_info);

```

Shrnutí priorit

Priorita	Problém	Dopad	Náročnost opravy
Střední	Trace(H) pro neuniformní mřížky	GCV může vybrat suboptimální λ	Vysoká
Nízká	Dělení nulou	Teoretický edge case	Nízká
Nízká	Magic numbers	Údržba kódu	Nízká
Nízká	I/O oddělení	Integrace do jiných systémů	Střední
Nízká	Přesnost derivací	Mírně horší derivace na okrajích	Nízká

Závěr

Po opravách V4.7 je modul `tikhonov.c` funkčně korektní pro většinu praktických případů. Zbývající problémy jsou převážně:

1. **Numerické edge cases** — `trace(H)` aproximace pro silně neuniformní mřížky
2. **Kvalita kódu** — dokumentace konstant, oddělení I/O
3. **Možná rozšíření** — přesnější derivace, váhované regrese

Žádný ze zbývajících problémů nepředstavuje kritickou chybu, ale jejich řešení by zlepšilo robustnost a udržitelnost kódu.

Doplňující audit — stav V5.0 (2. řád penalty)

Datum: 2026-05-30 **Stav:** Analýza po přechodu na true 2nd-order penalty $(D^2)^T W D^2$ (V5.0/2026-02-07) **Rozsah:** Nové nálezy nad rámec auditu V4.7 výše. Body z V4.7 (`trace` na neuniformních mřížkách, dělení nulou, magic numbers, I/O, derivace, váhy) zde neopakuji — stále platí.

A1. Nekonzistentní význam λ mezi diskretizacemi AVERAGE/LOCAL

Závažnost: Vysoká **Umístění:** `build_band_matrix()` ř. 75-134, `compute_functional()` ř. 191-218

Obě větve penalizují jiný funkcionál:

- **AVERAGE** ($CV < 0.15$): matice je λ/h^4 · stencil, penalizuje prostý součet $\lambda \sum_i (u_i'')^2$ — **bez míry** dx .

- **LOCAL** ($CV \geq 0.15$): váha $w_k = h_{\text{sum}}/2$, penalizuje $\lambda \sum_i (u_i'')^2 h_i$ — **aproximaci integrálu** $\lambda \int (u'')^2 dx$.

Protože $\sum (u'')^2 \approx \frac{1}{h} \int (u'')^2 dx$, platí fakticky $AVERAGE \approx LOCAL / h_{\text{avg}}$. Tedy:

- Při překročení prahu $CV = 0.15$ se efektivní síla **stejného** λ skokově změní o faktor $\sim 1/h_{\text{avg}}$ (nespojitosť chování na hranici).
- Pokud data nejsou normalizovaná na $h_{\text{avg}} \approx 1$, hledá `find_optimal_lambda_gcv` λ pro jiný funkcionál, než jaký se reálně řeší.

Doporučení: sjednotit měřítko — buď v $AVERAGE$ nést váhu h_{avg} ($\text{coeff} = \lambda/h^3$ místo λ/h^4), nebo explicitně zdokumentovat jako záměr.

A2. L-curve používá $\lambda \cdot \|D^2 u\|^2$ místo seminormy $\|D^2 u\|^2$ — □ **OPRAVENO (v5.11.32)**

Závažnost: Střední-vysoká **Stav:** Opraveno ve v5.11.32 — `find_lambda_lcurve()` nyní loguje `regularization_term / lambda` (čistá seminorma $\|D^2 u\|^2$), s ošetřením $\lambda = 0$.

Umístění: `find_lambda_lcurve()` ř. 448-449

`reg_vals[i] = log(regularization_term)`, kde `regularization_term` už obsahuje faktor λ (ř. 220: `*reg_term *= lambda`). Standardní L-křivka je parametrická křivka ($\log \|residual\|^2$, $\log \|Lu\|^2$) se seminormou **bez** λ . Přimíchaná monotónní složka $\log \lambda$ deformuje křivost a posouvá detekovaný roh.

Oprava (triviální): logovat `regularization_term / lambda` (a ošetřit $\lambda=0$).

A3. GCV trace neodpovídá reálně řešené matici (LOCAL)

Závažnost: Střední **Umístění:** `compute_gcv_score_robust()` ř. 380-387

Trace vždy používá vlastní čísla $AVERAGE$ varianty $(4 \sin^2(\theta/2)/h_{\text{avg}}^2)^2$, i když pro $CV \geq 0.15$ solver běží v **LOCAL** větvi. Pro neuniformní mřížku se tak GCV počítá k **jinému operátoru**, než jaký se invertuje — strukturální nesoulad, ne pouhá nepřesnost (rozšiřuje bod 1.1 z auditu V4.7).

Navíc komentář „eigenvalues of $(D^2)^T D^2 = (\text{eigenvalues of } D^1{}^T D^1)^2$ “ platí jen pro Fourierův symbol ve vnitřku; jako operátorová identita **neplatí** a uzly $\theta = \pi k/n$ neodpovídají přirozeným OP této matice.

A4. Ad-hoc exponenciální penalta degraduje GCV — □ **OPRAVENO (v5.11.33)**

Závažnost: Střední **Stav:** Opraveno ve v5.11.33 — penalta GCV `*= exp(10*(tr(H)/n - 0.7))` odstraněna, `-l auto` nyní minimalizuje učebnicový GCV. README sekce

„Enhanced GCV“ zrušena, diagnostický štítek „pGCV“ → „GCV“. **Umístění:** compute_gcv_score_robust()

gcv_score *= exp(10*(trace_ratio - 0.7)) **opouští teorii GCV.** GCV je principiální kritérium; ruční exponenciála systematicky tlačí k over-smoothingu bez statistického opodstatnění. Pokud GCV vybírá příliš malé λ , příčina je pravděpodobně v nepřesném trace (A3) — tato záplata to jen maskuje.

Empirické ověření před odstraněním: na testech (sinus + bílý i AR(1) korelovaný šum, $n = 60-500$) se vybrané λ s penaltou vs. bez ní lišilo **nanejvýš o jeden krok hledací mřížky**. Jmenovatel GCV $(1 - \text{tr}(H)/n)^2$ sám trestá $\text{tr}(H) \rightarrow n$, takže penalta byla z velké části redundantní a aktivovala se jen zřídka. Praktický dopad odstranění je tedy zanedbatelný a přínosem je jednodušší, designově čistý kód.

A5. Zavádějící chybová hláška u dpbsv — ☐ OPRAVENO (v5.11.33)

Závažnost: Nízká **Stav:** Opraveno ve v5.11.33 — hláška u info > 0 už nenavrhuje „Try larger lambda“; nově vysvětluje, že $I + \lambda(D^2)^T D^2$ je pro $\lambda \geq 0$ vždy SPD, takže info>0 ukazuje na numerickou ill-conditioning. **Umístění:** tikhonov_smooth()

Matice $I + \lambda(D^2)^T D^2$ je pro $\lambda \geq 0$ **vždy** SPD (semidefinitní penalta + identita). info > 0 proto prakticky nemůže nastat kvůli „příliš malé λ “ → hláška „Try larger lambda“ je matoucí; reálné info > 0 by signalizovalo numerický/paměťový problém.

A6. Drobnosti (efektivita / čistota) — ČÁSTEČNĚ OPRAVENO (v5.11.33)

Závažnost: Nízká

- ☐ **Redundantní přepočítání** (opraveno v5.11.33): compute_gcv_score_robust už nepřepočítává h_min/h_max/ratio ručně, používá grid_info->ratio_max_min.
- ☐ **Mrtvý výpočet** (opraveno v5.11.33): h4/scale přesunuty do větve $n > 5000$, kde se jediné používají.
- ☐ **Opakovaný výpočet vlastních čísel:** $\sim 21 \times O(n)$ volání sin pro každého kandidáta λ , přitom vlastní čísla závisí jen na mřížce — předpočítat jednou. (Ponecháno — strukturální perf změna, ne čistý cleanup.)
- ☐ **Redundantní memset** po calloc u volajícího; komentář „useful if reused“ neodpovídá realitě. (Ponecháno — defenzivní, neškodné.)
- ☐ **compute_derivatives:** centrální difference $(y[i+1]-y[i-1])/(x[i+1]-x[i-1])$ je na neuniformní mřížce jen **1. řádu i ve vnitřku** (audit V4.7 zmiňuje jen okraje).

A7. Slabě penalizované okrajové body

Závažnost: Nízká (známé omezení) **Umístění:** penalty smyčky $k = 1 \dots n-2$

Přirozené OP (žádné řádky D^2 pro koncové body) znamenají, že $u[0]$ a $u[n-1]$ vstupují jen do penalty řádků $k=1$ resp. $k=n-2$. Koncové body proto zůstávají prakticky ne-

vyhlazené — klasický artefakt natural-spline analogie. Pokud uživatelům vadí šum na okrajích, příčina je zde.

Shrnutí priorit (V5.0 doplněk)

#	Problém	Závažnost	Stav
A1	Nekonzistentní význam λ mezi AVERAGE/LOCAL	Vysoká	☐ Otevřeno
A2	L-curve loguje $\lambda \cdot \ D^2u\ ^2$ místo $\ D^2u\ ^2$	Střední-vysoká	☐ Opraveno (v5.11.32)
A3	GCV trace neodpovídá LOCAL matici	Střední	☐ Otevřeno
A4	Ad-hoc exp penalta v GCV	Střední	☐ Opraveno (v5.11.33)
A5	Zavádějící hláška dpbsv	Nízká	☐ Opraveno (v5.11.33)
A6	Redundance / mrtvý kód / efektivita	Nízká	● Částečně (v5.11.33)
A7	Slabě penalizované okraje	Nízká	☐ Otevřeno (známé omezení)

Závěr V5.0: Funkčně je modul (mimo GCV na neuniformních mřížkách) korektní a paměťově čistý (goto cleanup bez leaků). Z malých problémů vyřešeno A2, A4, A5 a část A6. Zbývají prioritně **A1** (měřítko λ) a **A3** (trace pro LOCAL), protože ovlivňují výběr λ i výsledek na neuniformních mřížkách.